이동형

DevOps Engineer | 경력기술서

GitHub: github.com/somaz94 Blog(EN): somaz.blog Blog(KR): somaz.tistory.com Portfolio: somaz.blog/portfolio

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- **EKS·GKE 비용 최적화** - 하이브리드 전환으로 20%, GCP 마이그레이션(Fargate → GKE 노드 풀 + MSP 할인)으로 50% 절감
- **그린필드 EKS 단독 구축·런칭** - 신규 퍼블리싱 게임 AWS 프로덕션, 플랫폼 100% GitOps(IAM 제외), IRSA·Pod Identity 정적 키 없는 인증
- **IaC 표준화** - 온프레미스·AWS·GCP를 Terraform·Ansible로 코드화, 신규 컴포넌트 IaC 100%(IAM 제외), 변경 이력 100% Git 추적, 동일 파이프라인을 타 프로젝트에 빠르게 재구축
- **AI 워크로드 GPU 인프라** - GCP A100·L4 GPU VM을 Terraform으로 관리(너디스타), 온프레미스 GPU에 사내 LLM 추론 운영(팬텀)

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 향상

- **파이프라인 설계·고도화** - GitHub Actions·GitLab CI·Jenkins·ArgoCD 멀티클러스터 배포 자동화
- **배포 생산성·신뢰성** - 배포 시간 67% 단축, GitOps 지속 배포로 시장 대응 가속
- **Canary 무중단 롤아웃** - Blue-Green → Argo Rollouts Canary 전환, promote 시 503 에러 제거
- **CI 빌드 환경 표준화** - 공용 이미지 계층화·버전 갱신 자동화, 준비 5분 → 약 10초(약 97% 단축), 버전 일관성 100%

모니터링 및 관측성 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- **중앙 집중식 로깅 3단계 고도화** - ELK → EFK → ECK Operator, Elastic Stack 9.x CRD 선언적 관리(TLS 자동 회전)
- **메트릭 모니터링** - kube-prometheus-stack 알림·Grafana 대시보드, exporter(MySQL·Redis·Elasticsearch·PostgreSQL) 통합
- **분산 추적** - OpenTelemetry + Tempo로 Metrics ↔ Traces ↔ Logs 3-signal 확보, 샘플링 분리로 정확도 유지·비용 절감

자동화 및 팀 역량 강화

Python·Bash 운영 자동화 도구 구축과, 파트너사 엔지니어가 스스로 운영하도록 만드는 교육·자료 표준화

- **운영 자동화 재구축** - Shell → Python(표준 라이브러리) 이관, 단계별 자동 배포·차트 업그레이드 MR 자동 생성
- **엔지니어링 교육** - 파트너사(Delivery Partner) 엔지니어 대상 Kubernetes 6개월 실습 설계·운영(평균 10명·회당 4시간), 자료 표준화·재사용
- **개인 프로젝트 공개 운영** - 공개 API 10여 곳에서 평일 하루 2회 수집·검증·재배포(경제 대시보드), 백엔드 없는 다이어그램 에디터

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

네트워크 스택 전환, 공용 차트 오픈소스 공개, 업스트림 기여

- **Gateway API 마이그레이션** - Ingress-nginx → NGINX Gateway Fabric 단일 컨트롤플레인, 사고 없이 전환·wildcard TLS 통합
- **오픈소스 공용 자산 배포** - Helm 차트·Kubernetes CRD 컨트롤러·Composite GitHub Actions·Ansible 컬렉션 다수 공개(ArtifactHub·GitHub Marketplace·Ansible Galaxy)
- **오픈소스 업스트림 기여** - apache/airflow·OpenTelemetry·prometheus-community Helm 차트에 Gateway API HTTPRoute 지원, kubernetes-sigs(Kueue·KWOK)·Prometheus Operator·k3s·Gitea에 버그 수정·기능 개선
- **NGINX Gateway Fabric 업스트림 기여** - 정책 상태 조건 구현과 복잡한 컨트롤러 함수 11개 리팩토링으로 NGF 2.7 릴리스 공지에 커뮤니티 기여자로 등재

인프라 보안 & IAM

Keycloak Operator SSO와 Vaultwarden으로 사내 인증·시크릿 거버넌스 일원화

- **중앙 인증(SSO/OIDC)** - Keycloak Operator로 GitLab 계정 단일 로그인, ArgoCD·Harbor·Vaultwarden OIDC 연동 완료
- **사내 시크릿 거버넌스** - Vaultwarden을 Kubernetes에 Helm 배포, GitLab SSO·자동 백업·대화형 restore 스크립트
- **GitOps 시크릿** - 평문 DB 비밀번호 → External Secrets Operator + AWS Secrets Manager, pull secret → Sealed Secrets

주요 경력 (Professional Experience)

팬텀(콘크릿트 스튜디오) 2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, dev·qa는 온프레미스에, review·prod는 AWS에 두는 하이브리드 아키텍처를 설계·구축·운영해 월 인프라 비용을 20% 절감했습니다.
라이브 게임은 외부 퍼블리셔와 협업해 2차 기술지원·운영을 담당하고, 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도: 하이브리드 아키텍처 설계·구축 100% (단독) / 라이브 게임 2차 기술지원 및 운영은 외부 퍼블리셔 운영팀과 협업 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS 단일 환경에서 운영하며 클라우드 비용 부담이 컸습니다. 트래픽 변동이 적은 비프로덕션 환경까지 클라우드 리소스를 상시 점유하고 있어 비용 최적화와 환경 분리가 필요했습니다.

접근 전략

- **하이브리드 아키텍처 설계** - 워크로드별 비용 및 사용 패턴을 분석해 dev·qa 환경은 온프레미스로 전환하고, 확장성과 고가용성이 필요한 review·prod 환경은 AWS에 유지
- **네트워크 및 파이프라인 격리** - 복잡한 사이트 간 VPN 구성 없이 애플리케이션 레벨에서 트래픽을 분리하고, 이미지 레지스트리도 이원화(dev·qa=Harbor, review·prod=ECR)하여 온프레미스와 AWS가 각자의 레지스트리로 빌드·운영
- **IaC 기반 운영 표준화** - 온프레미스와 AWS 전반에 Terraform과 Ansible을 도입해 신규 컴포넌트 IaC 100%(IAM 제외)를 달성하고, 이종 인프라 환경에서도 일관된 형상 관리 체계 수립
- **IAM 자격증명 거버넌스** - 서비스·CI 계정만 코드로 관리하고 콘솔 계정은 분리. Terraform state 파일에 장기 자격증명(액세스 키)이 평문으로 남지 않도록 사용자 키 자동 생성을 배제하고 개별 발급·회전 정책 적용

트러블슈팅

- **클라이언트 버전별 백엔드 분기 라우팅** - EKS 단일 Ingress 환경에서 구버전과 신버전 게임 클라이언트 요청을 서로 다른 백엔드로 분기해야 하는 요구사항 발생.
  ALB HTTP 커스텀 헤더 기반의 조건부 라우팅 규칙과 타깃 그룹 매핑을 조합해 추가 인프라 구성 없이 유연한 분기 라우팅 구현
- **롤링 배포 시 L7 커넥션 단절 방지** - ALB 타깃 등록 해제 지연(deregistration delay)과 SIGTERM 타이밍이 어긋나 종료 중인 Pod로 신규 요청이 유입되거나 진행 중인 요청(in-flight)이 끊김. preStop 훅으로 헬스체크를 먼저 실패시켜 신규 유입을 차단하고 드레인 지연 동안 잔여 트래픽을 소진, RollingUpdate는 Pod 우선 생성(add-before-remove)으로 조정해 롤아웃 중 용량 유지 - 커넥션 단절 0건.
  CloudWatch 기반 Grafana에서 ELB 5xx·Target 5xx를 분리해 배포 전후 대조, 5xx 0건·p95 유지 확인

성과

- 워크로드 분석을 근거로 한 하이브리드 재배치로 월 인프라 비용을 20% 절감하고, 절감분은 신규 프로젝트에 재투자
- 개발·QA와 프로덕션 환경을 물리적으로 분리해 장애 영향 범위를 해당 환경으로 제한

프로젝트 2: 게임 애플리케이션 CI/CD 파이프라인 구축 및 CI 빌드 환경 표준화 (App GitOps)

기여도 100% (파이프라인·CI 표준화 전 과정 단독 설계·구축) 2024.12 ~ 현재 (운영 중)

게임 애플리케이션의 수동 빌드·배포를 GitLab CI와 ArgoCD 기반 GitOps 파이프라인으로 자동화했습니다. 이어서 사내 저장소 전체가 공유하는 CI 도구 이미지 계층과 버전 관리 체계를 표준화해, 빌드 환경 준비와 도구 버전 관리를 공용 이미지와 중앙 버전 변수 한곳으로 모았습니다.

2-1. 게임 애플리케이션 배포 CI/CD 파이프라인 (App GitOps, 2024.12 ~ 현재)

문제

개발자가 수동으로 빌드 후 서버에 직접 배포하는 방식이라 배포당 평균 30분이 소요되었고 휴먼 에러로 인한 서비스 장애가 잦았습니다.

접근 전략

- **GitOps 기반 지속 배포(CD) 파이프라인 구축** - GitLab CI와 ArgoCD를 연계해 Git Push를 트리거로 빌드 → 테스트 → 배포 전 과정을 자동화하고, Git에 선언된 형상과 클러스터 상태 간 불일치를 감지해 자동 복구(Self-healing)하도록 구성. dev·qa·review·prod 4개 환경의 배포 로직을 단일 파이프라인으로 통합해 모든 환경에 같은 절차로 배포
- **최소 권한 기반의 안전한 이미지 빌드 환경 수립** - Docker-in-Docker(DinD)는 호스트 관리자 권한(Privileged 모드)이 필요해, 도커 데몬 없이 사용자 영역(User-space)에서 컨테이너 이미지를 빌드하는 Kaniko로 전환.
  빌드 컨테이너에서 Privileged 권한을 제거해, 빌드 잡이 침해되더라도 호스트까지 장악될 위험을 낮춤

트러블슈팅

- **ArgoCD 연쇄 삭제(cascading prune) 방지** - Git 커밋 오류로 하위 Application이 연쇄 삭제되지 않도록, 워크로드 Application은 `prune: true` + `selfHeal: true`로 자동 일치시키고 최상위 app-of-apps 루트·AppProject 계층은 `prune: false`로 두어 삭제는 수동 검증을 거치게 함.
  모든 sync 이벤트는 Slack 알림으로 실시간 추적
- **정적 데이터 배포 경로에서 SSH 키 제거** - CI 배포 계정이 SSH 키를 보유한 채 대상 인스턴스에 직접 접속(scp/rsync)하던 구조를, 산출물을 S3 중계 버킷에 올리고 SSM SendCommand로 인스턴스가 스스로 내려받는 방식으로 교체해 배포 환경의 정적 SSH 키를 완전히 제거하고 키 유출 경로를 없앰

성과

- 배포 시간 67% 단축 (30분 → 10분) - 빌드·배포·설정 수동 단계를 파이프라인으로 일원화해 휴먼 에러로 인한 배포 장애를 근본 원인부터 제거

- 데이터 배포 소스 페치 시간 약 78% 단축 (약 139초 → 약 30초) - 최신 커밋과 필요한 파일만 선별해 받는 Git 최적화(shallow + blobless fetch)로 전송 병목 해소
- 핵심 서비스 저장소당 주 평균 100회 배포 - GitOps 지속 배포로 Git push가 그대로 배포로 이어지고, 신규 기능이 프로덕션에 나가기까지 걸리는 시간도 단축
- 여러 저장소에 걸친 MasterData 파이프라인 구축 - 기획 데이터가 바뀌면 이를 쓰는 하위 저장소들의 배포 파이프라인을 자동 트리거

2-2. 공용 CI 도구 이미지 계층 구조 및 자동 버전 사이클 (2025.09 ~ 현재)

문제

개별 저장소(Repo)의 CI 잡마다 Alpine 베이스에 필수 도구(helm·helmfile·kubectl·crane·aws-cli)를 매번 설치해 빌드 환경 프로비저닝에 평균 5분이 소요되었습니다.
또한 도구 버전이 저장소별로 파편화되어 신규 버전 반영 시 전수 수정이 필요했고, `:latest` 태그라 이미지 안에 어떤 버전이 들어 있는지 알기 어려웠습니다.

접근 전략

- **Layer 1 (Mirror)**: 외부 레지스트리(Docker Hub, GHCR)의 도구 이미지를 멀티 아키텍처 매니페스트를 유지한 채 `crane copy`로 사내 Harbor에 미러링해 외부 의존성 제거
- **Layer 2 (Custom)**: Layer 1을 기반으로 사전 빌드된 공용 이미지 5종을 구성해 개별 저장소가 이미지 태그 지정만으로 도구를 즉시 활용하도록 표준화
- **버전 관리 체계 일원화**: 공용 빌드 템플릿과 중앙 버전 변수를 정의해 각 저장소가 개별 도구 태그를 직접 추적하지 않도록 함
- **공용 CI 템플릿 모듈화**: 저장소마다 복사해 쓰던 CI 잡을 공용 템플릿 저장소로 통합하고 `include` 방식으로 참조.
  공용 앵커와 주요 잡(정적 키 없는 AWS 인증, Kaniko 빌드, ArgoCD 배포, 백업)을 중앙에서 관리하고, 모든 저장소가 같은 템플릿을 참조하게 해 공통 잡이 저장소마다 달라지지 않게 함
- **자동 버전 사이클 구축**: Upstream 신규 버전 감지부터 각 저장소 전파까지 자동화하되, 파이프라인 실행과 Tier 2 MR 승인은 사람이 직접 확인하도록 남겨 안정성 확보:
  · (1) Upstream 신규 버전 감지 (주간 스케줄러)
  · (2) Layer 1 미러링 자동 MR 생성
  · (3) Layer 2 리빌드 및 중앙 버전 변수 동기화 자동 MR 생성
  · (4) Consumer 저장소 대상 일괄 자동 전파 (템플릿 제공 저장소 제외)

트러블슈팅

- **도구 간 버전 충돌로 인한 공용 이미지 빌드 실패**: 자동 버전 상향 중 helmfile 1.5.1이 요구하는 helm 최소 버전(3.18.6)이 기존 고정 버전(helm 3.16.4)보다 높아 빌드 파이프라인이 멈춤
- **원인 및 해결**: 도구 간 의존성은 upstream 릴리스 노트에만 나와 있어 미리 자동으로 감지할 수 없다고 판단. helm·helmfile·kubectl에 minor-guard를 도입해 동일 minor 내 patch 버전만 자동 상향되도록 통제하고, minor·major 변경은 호환성 검증 후 수동 승격하도록 분리(필요할 때 환경 변수로 한 번만 예외 적용)

성과

- **CI 프로비저닝 시간 약 97% 단축 (5분 → 약 10초)** - 사전 빌드된 표준 이미지 적용으로 빌드 지연을 해소하고 사내 저장소 전반의 도구 버전 일관성 100% 달성
- 4-Tier 버전 거버넌스 수립 - 빌드 인자부터 중앙 변수 관리, 컨슈머 분리, 버전 불일치를 잡는 lint까지 파편화된 버전 관리 지점을 단일 흐름으로 정돈
- 순차 업그레이드 파이프라인 상시 운영 - 컴포넌트를 위험도에 따라 티어로 나눠(T1 주간 / T2 격주 / T3 월간 + 사전 점검) 신버전 감지, Slack 알림, wave 순차 적용을 자동화하고, major 버전은 자동으로 올리지 않도록 막고 T3는 실패하면 통째로 되돌리도록 구성
- 신버전 전파 자동화 대상은 helm·helmfile·kubectl·crane·rclone - 파급이 큰 이미지 리빌드는 스케줄이 아니라 사람이 검토한 뒤에 실행되도록 구성
- 멀티 아키텍처 환경 지원 - `docker buildx`로 amd64·arm64 이미지 빌드 체계 마련

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석까지 전 과정 단독 설계·구축) 2024.11 ~ 현재

로그가 분산된 온프레미스 환경에서 중앙 집중식 로깅 체계를 3단계로 고도화하고, 로그 수집 계층의 데이터 무결성과 고가용성(HA)을 강화한 뒤, 축적된 EFK 파이프라인을 게임 유저 리텐션 분석 플랫폼으로 확장했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

문제

온프레미스 환경에서 각 서버와 컨테이너 로그가 물리적으로 분산되어 있어 Pod 재생성 시 로그가 유실되었으며, 통합 관측 체계가 없어 장애가 나면 노드와 Pod에 하나씩 접속해 로그를 찾아야 했습니다.

접근 전략

- **수집·가공 아키텍처 경량화 (ELK → EFK)**: 노드별 Filebeat 수집 + 중앙 Logstash 가공 구조에서 JVM 기반 Logstash 티어의 과도한 메모리 부하를 해소하기 위해, 수집기는 경량 Fluent-bit DaemonSet으로, 파싱은 Fluentd로 역할을 분리해 전환
- **선언적 클러스터 관리 전환**: 공식 Helm 차트가 2년 이상 정체돼 ECK Operator를 도입하고, CRD 기반의 선언적 라이프사이클 관리 체계 구축
- **클라우드(EKS) 확장 및 정적 키 없는 자동화**: 온프레미스 스택을 AWS EKS 환경에 맞춰 템플릿화해 배포하고, 인덱스 수명주기(ILM) 및 S3 스냅샷 백업 경로에 IRSA를 적용해 정적 자격증명 없이 자동화

트러블슈팅

- **ECK Admission Webhook 다운그레이드 거부로 인한 복구 장애**: 온프레미스 Elastic Stack 신규 버전 자동 갱신 중 릴리스 피드에는 등록되었으나 레지스트리에 미배포된 이미지를 참조해 `ImagePullBackOff` 발생.
  즉각 롤백을 시도했으나 ECK Admission Webhook의 다운그레이드 거부 정책으로 복구가 차단되며 Operator 장애로 확산
- **원인 분석 및 개선**: 진짜 위험은 배포 실패가 아니라 되돌릴 수 없는 상태라고 판단. 업그레이드 파이프라인에 레지스트리 이미지 존재 여부 사전 확인, 클러스터 헬스체크, 메이저 버전 상향 시 스냅샷 백업 경고, Webhook 제약을 고려한 롤백 절차를 넣어 재발 방지

성과

- 분산 노드 및 휘발성 Pod 로그의 중앙 집계 체계 구축 - Pod가 재생성·종료돼도 로그가 남게 하고, Pod마다 접속해 확인하던 로그를 Kibana 한 화면에서 시간순으로 추적하도록 구성
- 운영 자동화 및 Elastic Stack 메이저 버전 상향 - TLS 인증서 갱신·계정 프로비저닝·노드 교체 같은 수동 작업을 자동화(CRD 선언 기반 롤링 업그레이드)하고, 2년 이상 정체되었던 Elastic Stack 메이저 버전 업그레이드 완료 (8.x → 9.x)

3-2. Fluent-bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 현재)

읽은 위치를 보존해 로그 중복·누락 방지: 로그를 어디까지 읽었는지 기록하는 SQLite 오프셋 DB를 노드 로컬 스토리지에 저장해 수집기 Pod가 재시작돼도 읽은 위치를 기억하게 했고, 그 결과 로그 중복 수집은 0건이었습니다.

에러 없이 멈춘 수집 감지: 프로세스는 다운되지 않았는데 로그 수집만 멈춘 상태를 감지하는 모니터링과 알림을 추가했습니다.

재시도 없이 로그가 버려지던 문제 해결: Elasticsearch 노드를 하나씩 재시작하는 동안 Fluentd가 전송에 실패한 로그를 재시도도 알림도 없이 버리던 원인을 플러그인 코드에서 찾아, 모아 둔 로그를 한 요청으로 보내 실패하면 통째로 다시 보내도록 바꾸고 운영 설정 그대로 재현해 확인했습니다.

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

EFK 파이프라인의 제품 분석 플랫폼 확장: 장애 분석·검색용 로그 인프라를 확장해, Elasticsearch Transform(5분 주기)으로 인게임 이벤트를 사용자당 1행으로 집계했습니다.

리텐션 지표 시각화: 별도 상용 SaaS를 도입하지 않고 기존 EFK 파이프라인과 Kibana 대시보드만으로 D+1~D+30 리텐션과 코호트 분석 환경을 구축했습니다.

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버 · 셀프서비스 운영 콘솔)

기여도 100% (전체 시스템 설계 및 개발) 2025.02 ~ 현재

개발팀의 반복적인 수동 작업을 자동화하고 생산성을 높이기 위해 6가지 사내 도구를 설계·개발·운영했고, 마지막에는 이들을 단일 인증 게이트 뒤의 셀프서비스 콘솔 플랫폼으로 묶었습니다.

4-1. APK 배포 자동화: Slack 알림 봇 · 정적 파일 서버 (2025.02 ~ 현재)

Jenkins에서 APK 빌드가 끝나면 Slack으로 완료 메시지와 다운로드 QR을 자동으로 보내는 Python 봇을 Kubernetes에 배포해, QA 전달 시간을 90% 단축하고 버전 혼동을 없앴습니다. 기존 파일 서버는 유지보수가 끊기고 디렉터리 UI·검색·ZIP 일괄 다운로드도 없었습니다.
그래서 Go 기반 정적 파일 서버를 자체 개발해 오픈소스로 공개했고, NFS 스토리지를 연결한 자체 Helm 차트로 배포해 빌드 산출물을 이 서버로 전달하도록 바꿨습니다.

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

GitLab push를 트리거로 디자인 문서를 Google Drive 공유 드라이브에 rclone으로 단방향 동기화했습니다. 체크섬 비교로 변경분만 보내고 바뀐 디렉터리만 clone해 전송량을 줄였으며, 문서를 고칠 때마다 Drive에 직접 올리던 시간을 10분에서 2분으로 80% 줄였습니다.
GitLab이 단일 기준이라 Drive에서 직접 고친 내용은 다음 동기화 때 덮어씁니다.

4-3. 사내 LLM 서비스 구축 (4주, 2025.11)

온프레미스 GPU 서버에 Ollama·Open WebUI를 docker compose로 올려 사내 LLM 서비스를 4주 만에 구축했습니다. 코딩 모델을 GPU 메모리에 계속 올려 두어 요청마다 다시 불러오는 대기 없이 응답하게 했고, 현재 개발팀 공용으로 운영하고 있습니다.

4-4. Git 저장소 미러링 도구 개발 및 Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 현재)

CodeCommit·GitLab·GitHub 간 수동 동기화의 누락과 지연을 해소하기 위해 Go 기반 미러링 도구 git-bridge를 자체 개발해 오픈소스로 공개했습니다. 동기화를 자동화하고 실패 재시도와 Slack 알림으로 상태를 실시간으로 확인할 수 있게 했습니다.
강제 덮어쓰기·삭제 같은 유실 위험 작업 앞에는 확인 가드를 두고, 동기화 이력 콘솔의 조회 범위를 기존의 10배(최대 5,000건)로 넓혔습니다. 종료 시 진행 중인 동기화를 끝까지 마치는 드레인(Graceful Shutdown, 실측 중앙값 2.9초)으로 배포 중에도 동기화가 유실되지 않도록 했고, 현재 사내 신규·구 GitLab 두 인스턴스를 잇는 상시 동기화 경로로 운영하고 있습니다.

4-5. 사내 셀프서비스 운영 콘솔 플랫폼 구축 (2026.07 ~ 현재)

문제

계정 생성·삭제 같은 반복 운영이 인프라 담당자만 실행할 수 있는 셸 스크립트에 묶여 있어, 모든 요청이 한 사람을 거치는 병목이 생겼습니다. 또한 10개의 사내 계정 시스템이 파편화되어 있어 특정 사용자의 잔여 계정과 권한 현황을 통합 추적·감사하기 어려웠습니다.

접근 전략

- **단일 OIDC 인증 게이트웨이 구축**: 개별 애플리케이션의 인증 구현 부담을 제거하고, 포털 전면에 중앙 OIDC 게이트웨이를 배치해 인증된 트래픽만 백엔드로 전달되도록 설계(신규 서비스 추가 시 설정 등록만으로 즉시 연동)
- **사내 도구 및 운영 서비스 통합 포털화**: 기존 사내 웹 도구와 신규 운영 서비스 5종(계정·Keycloak·Harbor·Elasticsearch·VPN)을 포함한 총 7개 애플리케이션을 단일 진입점으로 통합 제공
- **계정 시스템 10개 연동 및 감사 로그 체계화**: 분산된 계정 시스템 10개의 프로비저닝 API를 콘솔에 연동해 통합 조회·제어를 지원하고, 모든 계정 변경 이력(생성·수정·삭제)의 작업 주체를 기록하는 감사 추적 체계 구축

트러블슈팅

- **CI 빌드와 GitOps 동기화 레이스 컨디션으로 생긴 다운타임**: 매니페스트 커밋 시점과 실제 컨테이너 이미지 빌드 완료 시점이 어긋나, ArgoCD가 레지스트리에 아직 없는 이미지 태그를 동기화해 `ImagePullBackOff` 발생.
  특히 RWO 볼륨을 사용하는 서비스(전체 8개 중 6개)의 `Recreate` 배포 전략 특성상 기존 Pod가 먼저 종료되며 전면 서비스 중단 초래
- **원인 분석 및 개선**: 동기화 실행 전 대상 이미지 태그를 실제로 pull해 보는 ArgoCD PreSync 훅을 구현.
  이미지가 준비될 때까지 ArgoCD가 기존 Deployment를 건드리지 않도록 제어해 이미지가 없어 전면 중단되던 문제를 제거(Harbor 운영 서비스 기준 중단 약 180초 → 정상 교체 약 20초)

성과

- **운영 병목 해소 및 셀프서비스 전환** - 6주간 단독으로 설계·개발해, 담당자 한 사람을 거쳐야 하던 계정·권한 운영 요청을 각 팀이 자기 권한 범위 안에서 콘솔로 직접 처리하도록 전환
- **중앙 거버넌스 뷰 및 단일 접점 통합** - 사내 핵심 도구 7개를 단일 OIDC 인증 게이트 뒤로 일원화하고, 계정 시스템 10개의 상태를 사용자 단위로 단일 화면에서 통합 조회·감사할 수 있는 환경 구축

프로젝트 5: Kubernetes 클러스터 · 사내 시스템 운영 고도화

기여도 100% (단독 설계 및 표준 절차를 자동화 스크립트로 코드화해 팀 내 재사용) 2025.12 ~ 현재

안정적인 클러스터 운영을 위해 모니터링과 Kubernetes 업그레이드·Helm 관리를 자동화하고, 네트워크 컨트롤러를 Gateway API로 전환했으며, 스토리지 병목 해소와 배포/업그레이드 파이프라인 정비까지 이어갔습니다.
여기에 더해 사내 시스템 전반의 백업·복구 체계와, 오래 정체되어 있던 사내 GitLab 메이저 업그레이드까지 함께 맡았습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

kube-prometheus-stack 기반 커스텀 알림과 Grafana 대시보드를 설계하고 DB·Node Exporter·Cilium 메트릭을 통합했습니다. 알림 규칙을 컴포넌트 그룹·심각도(severity)별로 나누고, 상위 알림이 울리면 하위 알림을 억제해 같은 장애의 중복 통보를 막았습니다.
이후 클러스터 밖에서 CRD가 삭제돼 Prometheus가 멈췄는데도 아무도 알아채지 못했습니다. 감시하는 쪽이 감시 대상과 같은 클러스터에 있어 함께 멈췄기 때문입니다. 그래서 클러스터 밖 Bastion의 cron watchdog이 5분마다 모니터링 스택이 살아 있는지 확인하게 했고, 클러스터 안 CronJob은 감시 대상과 함께 멈출 수 있어 제외했습니다.

5-2. Kubernetes 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

Kubespray 업그레이드 절차와 업그레이드 후 클러스터 상태 점검(노드·버전 일치·etcd·API Server·DNS·인증서 만료 등)을 스크립트화해 검증 시간을 90% 단축했습니다.
인증서는 만료일 점검과 갱신 스크립트를 두어 만료 전에 잡아내도록 했고, Helm 프레임워크와 표준 템플릿 동기화로 차트 버전 관리를 표준화했습니다.

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

온프레미스의 여러 ingress-nginx 인스턴스를 NGF 단일 컨트롤플레인과 클래스별 Gateway CR로 통합했습니다. MetalLB IP를 그대로 유지해 기존 DNS와 방화벽을 건드리지 않고 사고 없이 전환을 마쳤고(클래스당 30~60초), 인증서를 wildcard로 합쳐 수동 갱신 작업을 절반으로 줄였습니다.

5-4. 스토리지 성능 최적화: etcd / DB SSD 마이그레이션 및 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

빌드 중 GitLab Runner(buildx)의 디스크 I/O가 치솟아 HDD에 있던 etcd와 게임 DB가 초 단위로 느려지는 병목을 진단했습니다. 컨트롤플레인과 DB 워커는 SSD로 옮기고, 빌드 워크로드는 별도 물리 서버의 전용 VM으로 완전히 분리했습니다.
그 결과 게임 DB COMMIT 지연이 9~14초에서 ms 단위로 돌아왔고, 빌드 중 게임 접속이 막히거나 서비스가 느려지는 장애는 0건이 되었습니다.

5-5. Kubernetes 인프라 배포 자동화 및 멀티클러스터 GitOps(ArgoCD) 전환 (2026.03 ~ 현재)

문제

Shell/awk로 짠 인프라 모노레포는 YAML 조작과 환경별 분기가 복잡해 테스트를 자동화할 수 없었습니다. 배포도 CI 러너가 `helmfile apply`로 직접 밀어 넣는 Push 방식이라, 클러스터 상태가 Git과 어긋나는 드리프트를 잡지 못했고 멀티클러스터로 넓히기도 어려웠습니다.

접근 전략

- **Python 표준 라이브러리만 쓰는 툴링으로 리팩토링**: pip으로 따로 설치해야 하는 외부 패키지 없이 Python3 모듈로 나누고 모듈마다 단위 테스트(`unittest`)를 하나씩 붙여, 로컬과 CI 러너에서 똑같이 실행되는 구조 확보
- **Wave 단위 배포·업그레이드 자동화**: 컴포넌트 의존 관계에 따라 배포 순서를 6단계 Wave로 정하고, 차트 새 버전이 나오면 Values diff와 호환성 검증 결과를 담은 MR을 자동으로 여는 파이프라인 구축
- **ArgoCD pull-GitOps 컨트롤플레인 전환**: ApplicationSet(Matrix/Git Generator)으로 온프레미스·AWS 배포를 자동화하고, 기존 Helm 릴리스는 리소스를 다시 만들지 않고 ArgoCD 관리로 무중단 이관
- **GitOps 거버넌스 및 안전장치 수립**: 최상위 계층의 연쇄 삭제를 막고 AppProject 허용 목록을 적용해 장애 영향 범위를 제한

성과

- 온프레미스·AWS 멀티클러스터의 플랫폼 릴리스를 GitOps 컨트롤플레인 하나에서 선언적으로 관리
- 레거시 Shell 스크립트를 단위 테스트를 갖춘 Python 툴링으로 모두 옮겨 배포 안정성 확보
- 차트 업그레이드 MR 자동화와 Wave 배포로, 새 버전 확인과 배포 순서를 사람이 챙기던 운영 부담 감소
- Git에 선언된 상태로 되돌리는 자동 복구와 계층별 Prune 통제로, 대규모 삭제 사고로 번질 수 있는 경로 차단

5-6. 백업·복구(DR) 체계 구축과 백업 검증 (2026.08 ~ 2026.09)

사내 시스템 8곳과 NAS 경로 2곳의 백업·복구 체계를 구축했습니다. 검증 기준을 '백업 작업이 끝났는가'에서 '백업 파일이 끝까지 온전하게 쓰였는가'로 바꿨고, 그 과정에서 DB 덤프가 중간에 잘렸는데도 백업 작업은 종료 코드 0으로 정상 종료되던 결함을 찾아 고쳤습니다.
그대로 두었다면 실제 복원할 때 실패했을 백업입니다.

5-7. 사내 GitLab 이원화와 구 GitLab 21단계 메이저 업그레이드 (2024.10 신규 구축 / 2026.08 ~ 2026.09 업그레이드 6일)

입사 직후 베어메탈에 새 GitLab을 구축해 새 프로젝트 저장소로 쓰고, 필요한 기존 저장소는 GitLab 미러링으로 옮겼습니다. 컨테이너로 돌던 구 GitLab은 백업·외부 공개용으로 남기고, 직접 개발한 git-bridge로 두 인스턴스를 계속 동기화합니다.
이후 오랫동안 업그레이드하지 못한 구 GitLab을 GitLab 공식 Upgrade Path 도구의 경로대로 6일 동안 21단계로 나눠 올렸습니다. 다운타임은 첫 단계에서 약 6분 30초였고(한 번만 필요했던 볼륨 이동 포함), 업그레이드 후 야간 DR 복제본은 123G에서 39G로 68% 줄었습니다.

프로젝트 6: 인프라 보안 및 거버넌스 체계 구축

기여도 100% (설계·배포·SSO 연동) 2026.04 ~ 현재

도구마다 흩어져 있던 로그인, 시크릿 관리, 원격 접속 방식을 하나의 기준으로 정리했습니다.

6-1. 중앙 인증(SSO/OIDC) 통합 (Keycloak Operator, 2026.04 ~ 현재)

Harbor·ArgoCD·Vaultwarden이 각자 GitLab OIDC에 연결하던 로그인을 Keycloak Operator로 구축한 중앙 IdP 하나로 통합했습니다.
GitLab 그룹이 Keycloak Group Mapper를 거쳐 ArgoCD 역할로 이어지게 해, 권한은 한곳에서 관리하고 한 번의 로그인으로 여러 도구를 쓰는 SSO 환경을 구축했습니다.

6-2. 시크릿 거버넌스 수립: Vaultwarden · GitOps Secrets (2026.04 ~ 현재)

메일·메신저에 흩어져 있던 사내 공용 비밀번호를 Vaultwarden으로 옮겨 SSO 로그인으로 한곳에서 관리하도록 했습니다.
Git에 평문으로 커밋되던 DB 접속 정보는 External Secrets Operator가 AWS Secrets Manager에서 받아 오게 하고, Harbor pull secret은 Sealed Secrets로 암호화해 GitOps 저장소로 평문 자격증명이 들어가던 경로를 없앴습니다.

6-3. 사내망 보안 원격 접속 환경 구축 (OpenVPN PKI, 2026.07)

동시 접속 50명을 기준으로 사내망 VPN 게이트웨이를 온프레미스에 PKI 인증서 방식으로 구축했습니다. 사용자별 인증서 발급부터 폐기까지는 다시 실행해도 안전한 셸 스크립트로 자동화했습니다.

프로젝트 7: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (단독 설계·구축·운영) 2026.06 ~ 현재 (구축·운영 중)

신규 외부 퍼블리싱 게임의 상용 런칭을 위해 EKS 프로덕션 환경을 처음부터 혼자 구축했습니다(7-1). 이어서 분산 추적(7-2), Canary 무중단 배포(7-3), 게임 CDN 엣지 보안(7-6)을 더해 소프트 런칭(7-4)을 맞았고, 실제 트래픽이 들어온 뒤에는 SLI/SLO와 에러 버짓(7-5)을 붙였습니다.

7-1. 그린필드 EKS 프로덕션 인프라 구축 및 실트래픽 최적화 (2026.06 ~ 현재)

문제

회사의 기존 게임은 이미 AWS에서 서비스하고 있었지만, 제품마다 AWS 계정을 따로 두는 원칙에 따라 신규 외부 퍼블리싱 게임은 새 계정에서 해외 상용 런칭을 해야 했습니다. 그래서 기존 클러스터에 올리지 않고, 그 계정에 고가용성 EKS 프로덕션 환경을 처음부터 구축했습니다.

접근 전략

- **온프레미스+AWS 멀티클러스터와 GitOps 관리**: EKS 클러스터를 새로 만들고, 플랫폼 컴포넌트 전부를 ArgoCD GitOps로 선언해 온프레미스 클러스터와 같은 방식으로 관리
- **Karpenter로 워크로드별 노드 분리**: 두 노드 풀 모두 ARM 인스턴스로 두되, 게임 서비스는 세션 도중 노드가 회수되지 않도록 On-Demand에, CI 러너는 여러 ARM 인스턴스 패밀리의 Spot에 분산해 비용을 줄이면서 Spot 회수 영향을 낮춤
- **네트워킹·인증 자동화**: AWS Load Balancer Controller, ExternalDNS, wildcard ACM 인증서를 묶어 Ingress만 선언하면 ALB 규칙, DNS 레코드, HTTPS 적용까지 자동으로 이어지게 하고, 모든 ServiceAccount에 Pod Identity/IRSA를 적용해 클러스터 워크로드의 정적 IAM 키 제거
- **로깅·시크릿 기반 마련**: eck-operator로 HA EFK 스택(3노드 3-AZ, EBS gp3, SLM으로 S3 스냅샷)을, External Secrets Operator로 시크릿 연동을 구성해 새 클러스터에서 로그를 모으고 시크릿을 가져올 경로 확보
- **배포 경로에서 SSH 키 제거**: 클라이언트 아티팩트는 Jenkins → S3 → CloudFront로 배포하고, 서버 매니페스트는 S3 중계 버킷에 올린 뒤 SSM SendCommand로 Bastion이 EFS에 받아 가게 해 배포 과정에서 SSH 키를 쓰지 않음

트러블슈팅

- **Karpenter 노드 축소 중 게임 세션 보호**: do-not-disrupt 어노테이션은 Karpenter 자체의 노드 정리·AMI 교체만 막고 노드 장애 같은 회수는 막지 못해 쓰지 않음.
  세션이 Redis에 있어 Pod가 옮겨 가도 끊기지 않으므로, 종료되는 Pod는 처리 중인 요청을 끝내고 내려가게 하고, 서비스마다 Pod를 최소 하나 유지하며, 노드는 한 번에 하나씩만 비우도록 제한
- **롤링 배포 후 빈 노드가 회수되지 않던 문제**: 실사용 101MiB인 서비스에 메모리 Request가 2GiB로 잡혀 노드 점유율이 92%와 35%로 벌어지고 자동 축소가 일어나지 않던 원인을 규명.
  Request를 512MiB로 낮춰 노드 점유가 다시 고르게 채워지게 하고, 사용률이 낮은 노드도 정리하도록 Consolidation 범위를 넓힘
- **노드 교체가 같은 서비스의 Pod를 한 노드에 몰아넣던 문제**: AMI 드리프트로 노드가 교체될 때 한 서비스의 Pod 2개가 새 노드 한 대에 함께 올라가 단일 장애점이 되고, 그 서비스가 17초간 Degraded로 떨어짐. 노드 간 Pod 수 차이가 1을 넘지 않게 하고, 자리가 없으면 기존 노드에 올리지 않고 대기시켜 Karpenter가 새 노드를 띄우게 함. replica가 2개인 동안은 이 제약만으로 두 Pod가 항상 다른 노드에 배치됨.
  노드당 Pod 수를 제한하는 것이 아니라 노드 간 개수 차이만 보므로, HPA로 Pod가 노드 수보다 많아지면 한 노드에 여러 개가 배치됨. Canary 배포 중 새 버전 Pod가 기존 버전과 한 묶음으로 세어져 배포가 멈추지 않도록, 분산 계산 범위는 배포 버전 단위로 한정
- **Consolidation이 노드를 끝없이 교체하던 문제**: 위 분산 제약을 다른 서비스에도 걸자 Karpenter의 배치 판단과 실제 스케줄 결과가 어긋나, 약 7분 30초마다 노드를 지우고 곧바로 새로 띄우는 일이 16번 반복됨. 원인을 사용률 기반 정리의 시뮬레이션 불일치로 보고 빈 노드만 정리하는 정책으로 되돌림.
  운영 중인 Pod를 반복해서 축출하느니 사용률이 낮은 노드를 남겨 두는 편을 택했고, 스케일아웃이 아직 없는 소프트 런칭 규모라 그 추가 비용이 크지 않다고 판단
- **기동 중인 Pod로 트래픽이 들어가던 문제**: ALB가 아직 준비되지 않은 Pod에 트래픽을 보내 간헐적으로 5xx가 나던 것을 ALB Pod Readiness Gate로 해결. Target Group 헬스체크를 통과해야 Pod가 Ready로 판정되게 함

성과

- 신규 게임 전용 AWS 계정에 EKS 프로덕션 환경을 처음부터 구축하고, 플랫폼 매니페스트 100%를 GitOps로 관리(IAM 제외)
- Bastion SSH 키와 클러스터 워크로드의 정적 IAM 키를 제거하고, CI 에이전트에 남은 AWS 키는 역할 전환 권한만 갖도록 제한
- 프로덕션과 심사용 워크로드를 NodePool 하나로 합쳐, 풀을 나누면 풀마다 최소 1노드씩 드는 비용을 없앰
- GitOps 매니페스트와 CI/CD 전환 절차를 런북으로 문서화해, 다른 사람이 넘겨받거나 같은 환경을 다시 구축할 수 있게 함

7-2. OpenTelemetry 엔드투엔드 분산 추적 파이프라인 (OTel + Tempo, 2026.06 ~ 현재)

문제

기존 관측 체계(EFK 로깅, Prometheus 메트릭)만으로는 마이크로서비스 사이의 호출 흐름과 어느 구간에서 지연이 생기는지, 에러가 어느 서비스까지 번지는지를 알기 어려웠습니다.

접근 전략

- **추적 스택 구축과 자동 계측**: AWS EKS에 OpenTelemetry Collector와 Grafana Tempo(S3 백엔드, EKS Pod Identity로 연동해 정적 키 없음)를 구축하고, OTel Operator의 자동 계측으로 애플리케이션 코드를 수정하지 않고 언어별 SDK 주입(.NET 서비스는 arm64 노드를 지원하지 않아 보류 — 호출하는 쪽 span으로만 보임)
- **메트릭 추출 지점과 저장 지점을 분리**: RED 메트릭은 샘플링 전 100% Span에서 뽑아 집계 값이 실제와 어긋나지 않게 하고, 저장은 Tail Sampling(에러 전체 + 500ms 초과 요청 + 정상 요청 10%)만 남겨 저장 비용을 줄임

성과

- **3-Signal 상호 연계**: Metrics·Traces·Logs를 Grafana 한 화면에서 오갈 수 있게 연결해 서비스 간 호출 흐름과 병목 구간을 가시화하고, 트레이스에서 그 요청이 남긴 로그만 trace_id로 바로 열리게 함(기존 로깅은 프로젝트 3, 메트릭은 5-1)
- **2계층 라우팅으로 트레이스가 흩어질 수 있는 구조 수정**: Collector를 여러 replica로 운영하면 앱 Pod마다 gRPC 커넥션이 서로 다른 인스턴스에 고정됨. 한 트레이스의 Span이 Pod 경계를 넘으며 여러 인스턴스로 흩어지면 에러 트레이스 전량 보존과 Service Graph 조인이 함께 깨질 수 있다는 구조적 결함 식별
- 수신 계층(Layer 1)과 샘플링 계층(Layer 2)으로 분리하고 Headless Service + Trace ID 기반 라우팅(loadbalancing exporter)을 적용해, 한 트레이스의 Span이 항상 같은 샘플링 인스턴스에 모이도록 보장
- 실트래픽 검증: 커넥션이 고정돼 있던 동안에는 replica별 메트릭 시리즈가 1,650 : 541로 한쪽에 쏠려 있었음. Trace ID 해싱을 적용한 뒤에는 두 샘플링 인스턴스가 초당 Span 4.09 : 5.18로 균등하게 받음

- **샘플링 계층 알림 공백 보강**: 2계층 분리 후 기존 Collector 다운 알림이 샘플링 계층의 전면 장애를 감지하지 못하는 공백을 확인하고, 알림 4종(샘플링 계층 다운, 버퍼가 넘쳐 판정 전에 버려진 트레이스, 버퍼 임계 근접, 정책 평가 오류)을 신설해 아무 표시 없이 사라지는 트레이스를 감시

7-3. Argo Rollouts 기반 Canary 무중단 배포 전환 (Blue-Green → Canary, 2026.06 ~ 현재)

문제

기존 Blue-Green 배포는 promote할 때 ALB Target Group을 통째로 바꾸는데, 새 Pod가 등록되기 전에 기존 Pod가 먼저 빠지면 트래픽을 받을 Pod가 없어 그 사이 들어온 요청이 503으로 실패했습니다.

접근 전략

- **Canary 롤아웃 전환**: 단계별 트래픽 분배 없이 같은 ALB Target Group 안에서 새 Pod가 준비된 뒤에 구버전 Pod를 내리도록 바꿔, 준비된 Pod 수가 목표 아래로 내려가지 않게 함
- **Stateless 구조 전제**: 게임 서비스는 상태를 JWT와 Redis·DB(TypeORM)에 두는 stateless HTTP라, 전환 중 구버전과 신버전에 요청이 섞여도 결과가 같음을 확인.
  다만 두 버전이 함께 떠 있으면 안 되는 변경(API 스펙 변경 등)은 이 방식으로 배포하지 않고 정기점검 시간에 처리
- **배포 진입·종료 시점 맞추기**: ALB Pod Readiness Gate로 ALB에 정상 등록된 Pod만 Ready로 치고, 종료 시에는 ALB 등록 해제 대기(30초)보다 preStop(35초)을, preStop보다 전체 종료 유예(terminationGracePeriodSeconds, 60초)를 길게 잡아 ALB에서 빠진 뒤에 Pod가 내려가게 함 (SIGTERM에서 스스로 처리 중인 요청을 정리하는 .NET 서비스만 75초)
- **배포 전략 템플릿화·컨트롤러 HA**: 공용 Helm 차트에서 설정값으로 Deployment와 Rollout을 바꿀 수 있게 템플릿화해, 공개 트래픽을 받는 프로덕션 게임 메인 서비스에만 Canary를 적용하고(같은 게임의 내부 서비스는 일반 Deployment 유지), 컨트롤러는 replica 2개의 Leader Election과 PDB로 노드가 교체될 때 둘이 함께 내려가지 않게 함

성과

- 준비된 Pod 수가 목표 아래로 내려가지 않는 구조로 바꿔, 프로덕션 게임 배포 중 트래픽을 받을 Pod가 하나도 없는 구간과 그로 인한 503이 생기지 않게 하고, 하위 호환 패치는 무중단으로 배포

7-4. 소프트 런칭 운영 (2026.07 ~ 현재)

성과

- 소프트 런칭을 시작해, 소규모 실트래픽을 앞서 구축한 Canary 배포와 3-Signal 관측 위에서 처리
- 글로벌 정식 런칭에 필요한 관측·배포·오토스케일링(HPA·Karpenter)을 모두 갖춤(소프트 런칭 규모에서는 아직 스케일아웃이 트리거되지 않음)
- 소프트 런칭 2주간의 실트래픽을 근거로 Pod requests·limits를 다시 잡아, requests가 과도해 HPA가 스케일아웃하지 못하던 구조를 바로잡고 requests를 CPU 48%·메모리 71% 줄임

7-5. SLI/SLO·에러 버짓 정의 (2026.08)

문제

소프트 런칭으로 실트래픽이 들어오기 시작했지만, 서비스가 정상인지를 숫자로 판단할 기준이 없었습니다. 그래서 장애 여부를 사람이 보고 판단해야 했고, 안정화 작업의 우선순위를 정할 근거도 없었습니다.

접근 전략

- **측정할 수 있는 신호로만 SLI 정의**: 목표를 먼저 정하고 데이터를 맞추는 대신, 샘플링 전 전체 Span에서 뽑은 요청 지연시간이 실제로 계산되는지 확인한 뒤 SLI로 기록. SLO 목표치는 대시보드에서 바꿔 가며 비교하게 두고, 확정은 기준선이 쌓인 뒤로 미룸
- **가용성 SLI는 보류**: 게임 서버에서 예외가 나도 HTTP 200을 돌려줘, 약 790만 건의 요청 Span 중 에러로 잡힌 것이 0건이었음. 이대로 가용성 SLI를 만들면 장애 중에도 늘 100%로 보여 에러 버짓이 줄지 않으므로, 애플리케이션이 에러를 에러로 돌려주기 전까지 보류

성과

- 지연시간 SLI 기록 규칙과 SLI/SLO 대시보드를 만들어, 목표치와 지연 기준을 바꿔 가며 에러 버짓 소진을 비교할 수 있게 함
- 실측 분포로 목표 후보를 걸러, 늘 통과해 의미가 없는 'p95 2초 미만' 같은 목표를 후보에서 제외

7-6. 게임 CDN 엣지 보안: AWS WAFv2 적용 (2026.07 ~ 2026.09)

문제

런칭을 앞두고 인터넷에 그대로 노출된 게임 CloudFront 앞단에 WAF가 없어, 웹 공격이나 악성 IP를 걸러 낼 장치가 없는 것이 가장 큰 보안 빈틈이었습니다.

접근 전략

- **Count 모드로 먼저 관측해 오탐 방지**: AWS 관리형 룰 그룹 4종을 차단하지 않고 매칭만 세는 Count 모드로 배포해, 실트래픽에서 규칙마다 무엇이 걸리는지 확인.
  오탐이 보이면 해당 세부 규칙만 예외로 두고, 확인이 끝나면 tfvars 한 줄로 차단 모드로 전환하게 설계해 검증 없이 차단해 정상 유저가 막히는 일을 피함
- **WCU 용량과 비용 산정**: 관리형 룰 4종의 용량을 1,125 WCU로 기본 상한 1,500 안에 두고, 비용을 현재 트래픽에서 월 약 $9, 본격 트래픽에서 월 약 $70으로 미리 산정
- **Rate-based 규칙은 보류**: 모바일 통신사는 CGNAT으로 여러 유저가 공인 IPv4 하나를 나눠 쓰는 경우가 많아 IP 기준 요청 제한이 정상 유저를 막을 위험이 있고, 당시 피크가 5분당 28건이라 임계값을 정할 데이터도 없었음.
  대신 트래픽이 쌓이면 IP 분포를 보고 정하고, IP 대신 세션 헤더를 기준으로 세는 방식도 검토하도록 문서화

- **내부 경로는 처음부터 차단**: 관리형 룰이 관측만 하는 동안에도, 운영자 허용 IP 밖에서 내부용 API 경로로 들어오는 요청은 별도 규칙으로 바로 차단

성과

- 게임 CDN 앞단에 WAFv2를 정상 유저에게 영향을 주지 않고 도입하고, 관리형 룰 4종을 실트래픽으로 관측하면서 차단 전환 절차와 Rate-based 규칙의 도입 조건을 문서로 정리

프로젝트 8: EKS Auto Mode 2차 클러스터 및 Site-to-Site VPN(WireGuard) 게이트웨이 구축

기여도 100% (단독 설계·구축) 2026.08 ~ 현재 (구축·전환 중)

문제

신규 퍼블리싱 게임은 퍼블리셔 내부 인증 계정과 퍼블릭 인터넷을 거치지 않는 프라이빗 경로로 통신해야 해 VPC 피어링이 필요했습니다. 그런데 기존 프로덕션 VPC 대역이 피어링 요건과 맞지 않아 VPC부터 클러스터·데이터 계층까지 새로 만들어야 했고, 퍼블리셔는 새 클러스터를 EKS Auto Mode로 구성해 달라고 요청했습니다.
기존 클러스터는 관리형 노드그룹 위에 Karpenter를 직접 운영해 컴퓨트 쪽 Terraform 관리 대상이 많았고, 노드 수명주기·Spot 중단·핵심 애드온도 직접 관리해야 했습니다. 또 VPC 피어링은 다른 망을 거쳐 가는 경로를 지원하지 않아 온프레미스에서 내부 ALB로 가는 프라이빗 경로를 따로 만들어야 했습니다.

접근 전략

- **EKS Auto Mode로 컴퓨트 계층을 관리형으로 전환**: 노드 프로비저닝과 수명주기, Karpenter 운영에 필요한 AWS 리소스(IAM·SQS·EventBridge), 내장 애드온(LB Controller·EBS CSI)을 AWS 관리에 맡기고, 코드에는 NodePool 선언만 남김
- **NodePool을 ArgoCD보다 먼저 배포하는 부트스트랩 순서 설계**: 모든 NodePool을 Graviton(arm64)으로, 빌드 풀은 Spot으로 쓰려고 amd64·On-Demand로 고정된 내장 general-purpose 풀을 끄자, 남은 system 풀의 `CriticalAddonsOnly` 테인트 때문에 자체 NodePool이 생기기 전에는 일반 Pod가 올라갈 노드가 없음.
  NodePool까지 ArgoCD로 배포하면 클러스터 재구축 시 ArgoCD부터 Pending에 멈추므로, NodePool·IngressClass는 ArgoCD보다 먼저 helmfile로 배포하는 Tier 0 단계에 남김
- **IPAM 제약에 맞춘 재설계**: 할당받은 /18을 /20 4개가 정확히 채워 서브넷을 늘릴 수 없어, 데이터 계층을 전용 서브넷으로 떼어 두던 구조 대신 AZ별 private 서브넷 안에서 보안 그룹으로 격리하도록 바꾸고, Aurora·Valkey·EFS를 이 구조로 옮기는 코드를 전환에 앞서 작성
- **Site-to-Site WireGuard VPN 게이트웨이 구축**: 퍼블릭 ALB를 하나 더 두면 같은 서비스가 호스트명·인증서·DNS 레코드를 따로 갖게 되고 환경이 하나 늘 때마다 그 세 가지가 함께 늘어나므로, 대신 터널링을 택해 단일 도메인·인증서 체계를 유지한 채 모든 내부 트래픽이 프라이빗 경로로 통신하도록 설계.
  온프레미스에서 AWS(EIP) 방향 아웃바운드로 터널을 수립하고, Peer Endpoint를 비워 첫 핸드셰이크 과정에서 상대 주소를 동적으로 학습하도록 구성
- **VPC 정적 라우트를 사무실 라우터 한 곳에서 관리**: 노드마다 라우트를 넣으면 설정이 빠진 노드의 Pod만 VPC에 닿지 못해 Pod 위치에 따라 성공과 실패가 갈리는 부분 장애가 생기므로, 라우트를 라우터에만 두어 경로 문제가 모든 호스트에서 한꺼번에 드러나게 함
- **Terraform 밖 의존 요소 관리**: `terraform plan`으로 감지되지 않는 터널의 외부 의존 요소 세 가지(라우터 정적 경로·게이트웨이 설정 파일·개인키)를 런북에 문서화.
  개인키는 state에 평문으로 남지 않도록 Terraform 밖에서 AWS SSM Parameter Store SecureString으로 생성하고 게이트웨이가 부팅 시 조회
- **플랫폼 컴포넌트 호환 마이그레이션**: 관측성·보안·CI 도구 등 16개 핵심 플랫폼 컴포넌트를 Auto Mode 사양에 맞게 재구성하고, 10개 컴포넌트를 ArgoCD pull-GitOps 관리 체계로 전환

트러블슈팅

- **Auto Mode가 ALB 그룹 어노테이션을 무시하는 문제 해결**: Auto Mode 내장 컨트롤러는 여러 Ingress를 ALB 하나로 묶는 `group.name` 어노테이션을 읽지 않고 `IngressClassParams`로만 그룹을 인식함. ALB stack 태그에 그룹명 대신 Ingress 이름이 붙어 있는 것을 보고 알아챘고, 그대로 두었다면 service 그룹 Ingress 5개 중 4개와 공개 호스트가 ALB를 할당받지 못할 상황이었음.
  ALB 그룹마다 IngressClass·IngressClassParams를 선언함. 클래스를 바꾸면 ALB가 새로 만들어지는데 기존 ALB가 같은 이름을 쓰고 있어, Ingress를 먼저 지운 뒤 새 클래스로 옮김
- **NodeClass API 그룹 변경 및 미지원 애드온 분리**: Auto Mode에서 NodeClass API 그룹이 `karpenter.k8s.aws/v1` EC2NodeClass에서 `eks.amazonaws.com/v1` NodeClass로 바뀌어 기존 매니페스트와 호환되지 않음.
  스펙을 신규 CRD에 맞춰 다시 쓰고, Auto Mode에 없는 EFS CSI Driver는 별도 배포로 분리

성과

- 새 클러스터에서 컴퓨트 계층의 Terraform 관리 대상이 102개에서 17개로 83% 줄고, Karpenter 컨트롤러와 그 운영에 필요한 IAM·중단 이벤트 처리는 AWS가 운영하며 노드 풀 정책만 NodePool로 직접 관리
- 온프레미스·AWS 간 Site-to-Site VPN 게이트웨이를 구축해 내부 ALB의 공용 인터넷 노출 없이, 환경이 늘어나도 단일 내부 호스트명과 단일 TLS 인증서 체계를 그대로 유지

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임·블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며 AWS → GCP 대규모 클라우드 마이그레이션에서 인프라 부문을 총괄했습니다.
비용 최적화와 글로벌 네트워크 성능 확보를 목표로, 게임 점검 시간을 활용해 최소 다운타임으로 전체 서비스를 이전했습니다. 소스 저장소는 온프레미스는 GitLab, 프로덕션은 GitHub로 이원화해 관리했고, 정체돼 있던 베어메탈 GitLab을 서버 교체 없이 공식 경로를 따라 다단계(약 12 hop)로 업그레이드했습니다.
또한 MongoDB·CloudSQL·GA·Dune 등 멀티 소스 데이터를 BigQuery로 모아 Google Sheets까지 자동 갱신하는 데이터 수집 파이프라인을 구축해, 분석가가 매일 2~3시간 들이던 수동 수집을 자동 적재로 대체했습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 아키텍처 설계·마이그레이션 전략 수립 및 실행 총괄 70% (나머지 30%는 서버 개발팀의 애플리케이션 측 마이그레이션 협업), CI/CD 파이프라인 구축 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용이 계속 올랐고, 특히 상시 가동되는 게임 워크로드를 Fargate로 돌리며 발생하는 Pod 단위 과금이 큰 비중이었습니다. 비용 절감과 멀티 리전 게임 서비스를 위한 글로벌 로드밸런싱 확보를 목표로 GCP 마이그레이션을 추진했습니다.

접근 전략

- **3단계 순차 이전** - 1단계 개발 환경 GCP 구축·검증, 2단계 QA, 3단계 프로덕션
- **네트워크·인증** - Shared VPC(Host Project / Service Project 분리) 설계, GitHub Actions의 GCP 인증은 Workload Identity Federation으로 전환해 서비스 계정 키 발급·순환을 OIDC 단기 토큰으로 대체.
  DNS는 서브도메인 위임(Hosting.kr → AWS Route53 → GCP Cloud DNS)으로 사전 검증하고, 최종 NS 변경만 점검 공지 후 전환
- **컴퓨트 재배치** - AWS Fargate 워크로드를 GKE 표준 노드 풀로 이전. 상시 워크로드는 Pod 단위 과금이 VM 단위보다 불리해 동일 과금 구조인 GKE Autopilot 제외
- **스토리지 제약 우회·WAF** - Filestore SSD 최소 2.5TB 제약은 Compute Engine에 pd-balanced 1TB 디스크를 붙인 NFS 서버 직접 구축으로 우회, Cloud Armor로 리전 차단·IP 화이트리스트 웹 방화벽(WAF) 정책 구성
- **IaC·GitOps 파이프라인** - Terraform으로 GCP 인프라 IaC화(IAM 제외 100%), 이전 후 GitLab CI + ArgoCD GitOps CI/CD 파이프라인 구축, Helm 차트로 환경별 설정 표준화

트러블슈팅

- AWS ALB는 대상이 모두 헬스체크에 실패하면 막지 않고 전체에 트래픽을 보내지만 GCP 로드밸런서는 차단해, AWS에서는 드러나지 않던 헬스체크 실패가 GCP로 옮기자마자 트래픽 단절로 나타남. GKE가 Pod를 NEG로 직접 붙이는 구조에 맞춰 헬스체크와 백엔드 설정을 다시 잡아 해결
- GKE Ingress의 BackendConfig에 지정한 헬스체크 경로가 응답하지 않아 503이 발생. 그 경로가 항상 응답하게 하고 ManagedCertificate·FrontendConfig 설정과 맞춰 외부 트래픽을 정상화
- Cloud CDN으로 옮긴 뒤 HTTP → HTTPS 리다이렉트가 빠져 게임 클라이언트 파일 다운로드가 실패. HTTPS로 넘겨 주기만 하는 HTTP용 URL Map을 따로 두고, 80 포트 forwarding rule을 그쪽으로 연결해 해결

성과

- AWS → GCP 이전, Fargate → GKE 표준 노드 풀 전환, MSP 리셀러 할인을 합쳐 클라우드 비용 50% 절감
- 3단계로 전체 서비스를 이전 완료. AWS를 그대로 운영하면서 GCP에 같은 구성을 새로 만들어 두고, 최종 DNS 전환만 2시간 내외 점검 공지 후 실행해 다운타임을 줄임
- IAM을 제외한 GCP 인프라를 모두 Terraform으로 관리하고, 배포 이력은 Git 커밋으로 모두 추적
- Workload Identity Federation 도입으로 GitHub Actions의 GCP 서비스 계정 키 관리 부담과 키 유출 리스크 제거

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% (Python 스크립트 개발 및 운영) 3개월 (2024.01 ~ 2024.03)

문제

게임·블록체인 데이터를 수동으로 수집하다 보니 분석가가 매일 2~3시간을 소비했고, 휴먼 에러로 인한 데이터 누락도 빈번했습니다.

접근 전략

- **멀티 소스 BigQuery 적재** - MongoDB는 Dataflow Flex Template ETL, CloudSQL은 BigQuery 직접 connection, Google Analytics · Dune은 각각의 API(Dune은 별도 API Key)로 적재, 공통 스키마로 정규화
- **분석가용 Google Sheets 자동 갱신** - Python Cloud Function이 BigQuery 쿼리 결과를 Google Sheets API로 적재, Cloud Scheduler가 Daily(전일)·Monthly(전월) 두 주기로 자동 트리거
- **전체 인프라 Terraform IaC** - BigQuery Dataset · Cloud Function · Scheduler · Cloud Storage · Artifact Registry · IAM · Service Account를 코드로 관리

트러블슈팅

- CloudSQL → BigQuery 직접 connection(federated query)이 리전 불일치와 connection Service Account 권한 부족으로 실패. connection을 동일 리전에 생성하고 connection SA에 Cloud SQL Client 권한을 부여하여 해결
- Dune API의 execute → poll → fetch 비동기 실행 구조와 요청 한도로 결과 누락·타임아웃 발생. 실행 상태 폴링과 백오프로 결과 수신을 보장하고 API Key를 Secret Manager로 분리
- Daily 재실행 시 동일 레코드가 BigQuery에 중복 적재되는 문제 발생. 중복 제거 Cloud Function에 MERGE 및 ROW_NUMBER() 기반 정합성 로직을 적용해 적재를 멱등화
- BigQuery 결과를 Google Sheets API로 적재할 때 호출이 몰리면 분당 요청 한도(429)에 걸리고, 결과가 크면 한 번에 쓰지 못해 갱신이 실패. 결과를 1,000행씩 나눠 날짜별 시트에 쓰고, 호출 사이에 간격을 두며 한도를 넘으면 지수 백오프로 재시도하게 해 한도 안에서 동작하게 함

성과

- 분석가가 매일 2~3시간 들이던 수동 수집을 자동 적재로 대체하고, 수작업 입력 실수로 생기던 누락을 없앰
- 전체 인프라를 Terraform으로 관리해 같은 파이프라인을 다른 GCP 프로젝트 3곳에 빠르게 재구축하고, 변경 이력은 Git으로 모두 추적

프로젝트 3: AI 워크로드용 GPU 인프라 표준화 및 확장 (Windows → Linux · 세팅 자동화 · 사내 / GCP / 외부 IDC)

기여도 100% (인프라 설계·구축·운영) 6개월 (2024.02 ~ 2024.07)

문제

사내 GPU 서버가 Windows로 운영돼 이미지 생성 모델(Stable Diffusion 계열)을 돌릴 때마다 엔지니어가 실행 환경을 직접 세팅해야 했습니다. 드라이버·CUDA·프레임워크 조합을 손으로 맞추다 보니 같은 환경을 재현하기 어려웠고, 장비는 있는데 쓰기까지 시간이 걸렸습니다.

접근 전략

- **Windows → Linux 표준화** - 사내 GPU 서버를 Ubuntu 22.04로 전환하고, 세팅 스크립트 하나로 드라이버 스택(nvidia-driver-550 · CUDA 11.8 · cuDNN 8.6) 설치·동작 확인, 공유 모델 저장소(NFS) 마운트, GPU별 Stable Diffusion WebUI 기동까지 처리해 어느 서버에서나 같은 환경을 재현. LoRA 학습용 kohya_ss와 ComfyUI도 같은 구성에 배치
- **GCP 클라우드 확장** - A100 40GB와 L4 GPU VM을 Terraform으로 관리하며 수요에 맞춰 구성을 바꿔 운영하고, 최대 A100 4장과 L4 8장까지 확장. 리전·존별 GPU 가용성 확인과 쿼터 상향을 거쳐 앞선 마이그레이션의 Shared VPC 서브넷에 연결해 네트워크 표준 유지
- **외부 IDC 20대 증설·워크로드별 카드 분리** - 같은 세팅 스크립트로 GPU 서버 20대 구축.
  학습은 VRAM이 큰 카드가 필요하고 추론은 요청마다 독립적이라 GPU 수만큼 처리량이 늘어, 학습·대용량 작업은 GCP A100, 대량 추론은 IDC의 소비자용 RTX 40 시리즈로 나눠 배치

성과

- Windows에서 손으로 하던 세팅을 Linux와 세팅 스크립트로 대체하고, 그 스크립트를 사내·GCP·외부 IDC GPU 서버 전체의 표준 구축 절차로 사용
- GCP GPU VM을 Terraform으로 관리해 구성 변경과 재구축을 코드로 추적하고, 기존 GCP IaC 저장소에 그대로 편입
- GPU 자원을 사내 1대에서 사내 10대, GCP A100·L4, 외부 IDC 20대로 확장하고, 학습과 추론에 맞춰 카드 등급을 나눠 배치

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했습니다. 금융권·공공기관 프라이빗 클라우드 인프라를 설계·구축하고, 파트너사 엔지니어 교육을 직접 맡았습니다.

프로젝트 1: 고객사 맞춤형 PaaS 솔루션 구축

기여도 100% (인프라 설계 및 Kubernetes 클러스터 구축) 11개월 (2022.04 ~ 2023.03)

문제

고객사마다 요구하는 환경과 서버 스펙이 서로 달라, 표준화된 솔루션으로는 대응이 어려웠습니다.

접근 전략

- **HA 클러스터 설계** - 고객사 요구사항에 맞춰 Kubernetes 클러스터를 HA 구성으로 설계하고 OpenStack으로 가상머신 관리 환경 구축
- **통합 스토리지** - Ceph로 블록·파일·오브젝트 스토리지 통합 제공

트러블슈팅

- **계획 정지·기동 절차** - 고객사 정전 작업은 전원을 내렸다 올리는 일이라 무중단이 불가능해, 데이터가 깨지지 않게 멈추고 문제없이 다시 올라오게 하는 절차를 만듦. VM부터 멈추고 DB·메시지 큐를 거쳐 스토리지를 마지막에 끄며, 다시 올릴 때는 역순으로 진행.
  DB 클러스터(MariaDB Galera)는 한 대씩 줄여 최신 데이터가 남은 노드부터 다시 올리고, 스토리지(Ceph)는 노드가 꺼져도 장애로 보고 데이터 복구를 시작하지 않도록 미리 설정. 현장에서 풀지 못한 기동 단계 문제(메시지 큐 노드가 클러스터에 다시 붙지 않음, 볼륨 생성 실패)는 2차 기술지원으로 복구

성과

- 5개 고객사에 HA 구성 PaaS 솔루션 구축·납품 후 안정 운영
- Ansible로 서버 프로비저닝과 구성 관리를 자동화해, 고객사마다 같은 플레이북으로 서버를 구성하고 배포 시간을 줄임
- 정전 작업용 정지·기동 절차서를 현장 엔지니어에게 넘기고, 기동 중 생긴 문제의 조치사항도 같은 절차서에 반영

프로젝트 2: 파트너사 엔지니어 Kubernetes 운영 교육 프로그램 설계·운영

기여도 100% (교육 설계 및 진행) 6개월 (2022.06 ~ 2022.11)

문제

파트너사 엔지니어들이 Kubernetes·컨테이너 기반 인프라 운영 경험이 부족해, PaaS 솔루션 도입 후 자체 운영에 어려움이 예상됐습니다.

접근 전략

- **단계별 커리큘럼** - Kubernetes 기초부터 클러스터 운영·트러블슈팅까지 단계별 교육 커리큘럼을 설계하고 실습 환경을 구성해 실무 중심으로 운영
- **PaaS 스택 병행 교육** - OpenStack·Ceph 스토리지 운영 교육을 병행해 전체 PaaS 스택 운영 역량 확보

성과

- 6개월간 월 1회, 회당 4시간·평균 10명 규모로 교육하고 운영 가이드를 전달해 파트너사 엔지니어들이 Kubernetes를 스스로 운영하게 하고 교육 후 기술 문의를 줄였으며, 표준화한 교육 자료는 이후 다른 고객사 교육에도 재사용